Lecture 26.12: K-Nearest Neighbors (KNN) from Scratch — Multi-Dimensional Implementation & Plurality Voting

📌 Executive Summary

Complete vectorized OOP implementation of KNNClassifier, nearest index retrieval with np.argsort, plurality voting with np.bincount, and visualization.

🔬 Mathematical Formulation & Theory

1. Complete OOP KNN Implementation

Following the official lecture notebook:

```
class KNNClassifier:
    def __init__(self, k=3):
        self.k = k

    def _euclidean_dist(self, x1, x2):
        return np.sqrt(np.sum((x1 - x2) ** 2))

    def fit(self, X, y): # Lazy Learner
        self.X_train = X
        self.y_train = y

    def _predict_one(self, x):
        # 1. Compute distances to all training points
        distances = [self._euclidean_dist(x, x_train) for x_train in self.X_train]

        # 2. Extract indices of k smallest distances
        knn_indices = np.argsort(distances)[:self.k]

        # 3. Fetch labels of k nearest neighbors
        knn_classes = [self.y_train[i] for i in knn_indices]

        # 4. Plurality / Majority voting
        majority_class = np.argmax(np.bincount(knn_classes))
        return majority_class

    def predict(self, X):
```

```
        y_pred = [self._predict_one(x) for x in X]
        return np.array(y_pred)
```

2. Algorithmic Breakdown of Steps

1. `distances` : Computes pairwise Euclidean distance between the test point x and every stored vector in `self.X_train` .
2. `np.argsort(distances)` : Returns index permutation that sorts the array in ascending numerical order. Slicing `[:self.k]` yields the nearest neighborhood $\mathcal{N}_k(x)$.
3. `np.bincount(knn_classes)` : Counts occurrences of non-negative integer class labels.
4. `np.argmax(...)` : Identifies the label with the maximum vote tally (plurality vote).

💻 Python & NumPy Implementation

```
import numpy as np
import pandas as pd

class KNNClassifier:
    def __init__(self, k=3):
        self.k = k

    def _euclidean_dist(self, x1, x2):
        return np.sqrt(np.sum((x1 - x2) ** 2))

    def fit(self, X, y):
        self.X_train = X
        self.y_train = y

    def _predict_one(self, x):
        distances = [self._euclidean_dist(x, x_train) for x_train in self.X_train]
        knn_indices = np.argsort(distances)[:self.k]
        knn_classes = [self.y_train[i] for i in knn_indices]
        majority_class = np.argmax(np.bincount(knn_classes))
        return majority_class

    def predict(self, X):
        return np.array([self._predict_one(x) for x in X])

# Lecture Dataset
X_train = np.array([
    [1, 2],
    [2, 3],
    [3, 3],
    [6, 5],
    [7, 7]
])
```

```
y_train = np.array([0, 0, 0, 1, 1])

X_test = np.array([
    [2, 2],
    [6, 6]
])

knn = KNNClassifier(k=3)
knn.fit(X_train, y_train)
y_pred = knn.predict(X_test)
print("KNN Test Predictions for [[2, 2], [6, 6]]:", y_pred)
```

🎯 Key Takeaways & Best Practices

1. **First-Principles Transparency**: Building from scratch demystifies optimizer mechanics, memory layouts, and algorithmic failure modes.
2. **Vectorization vs Loops**: Always eliminate Python `for` loops across sample dimensions by relying on BLAS-accelerated NumPy array operations.
3. **Shape Integrity**: Maintain explicit assertions on input array ranks and shapes (`X.shape == (m, d)` , `y.shape == (m,)`) to prevent silent broadcasting bugs.